# Code Explanation

GA Prompt Optimiser for LLM Code Generation

Evolving System Prompts on the APPS Dataset

Group 1 — Computational Intelligence, Spring 2026

April 2026

Condensed walkthrough: every file, every function, every call chain.
Includes cross-file data-flow diagrams and a complete worked example.

# Contents


**1 Introduction & Architecture** **2**
- 1.1 What the Project Does . . . 2
- 1.2 The Eight Files . . . 2
- 1.3 Module Dependency Graph . . . 2
- 1.4 Two-Model Architecture . . . 3
- 1.5 Master Call-Flow (one generation) . . . 3

**2 `config.py` — Hyperparameters** **4**
- 2.1 Purpose & Design . . . 4
- 2.2 Key Constants Explained . . . 4

**3 `seeds.py` — Initial Population** **5**

**4 `dataset.py` — Loading & Splitting Problems** **6**
- 4.1 `load_problems(apps_train_dir)` → `list[dict]` . . . 6
- 4.2 `split(problems, evolution_size, holdout_size, seed=42)` → `(list, list, list)` 6

**5 `evaluator.py` — Code Extraction & Sandboxed Testing** **8**
- 5.1 Overview . . . 8
- 5.2 `extract_code(response: str)` → `str` . . . 8
- 5.3 `has_function(code: str, fn_name: str)` → `bool` . . . 8
- 5.4 `run_tests(code, fn_name, inputs, outputs, timeout=5)` → `float` . . . 9

**6 `cost_tracker.py` — API Cost Logging** **11**
- 6.1 Purpose & Location . . . 11
- 6.2 `CallRecord` — One API Call's Data . . . 11
- 6.3 `CostTracker` — The Singleton Accumulator . . . 11

**7 `llm.py` — All Claude API Calls** **13**
- 7.1 Overview . . . 13
- 7.2 `_call()` — The Core Wrapper . . . 13
- 7.3 `generate_code()` — Fitness Evaluation . . . 14
- 7.4 `crossover()` — LLM-Mediated Merge . . . 14
- 7.5 `mutate_inject()`, `mutate_delete()`, `mutate_rephrase()` . . . 14

**8 `ga.py` — The Genetic Algorithm** **16**
- 8.1 `Individual` — The Chromosome Dataclass . . . 16
- 8.2 `compute_fitness(individual, problems)` → `float` . . . 16
- 8.3 `tournament_select(population, rng)` → `Individual` . . . 17
- 8.4 `breed(parent_a, parent_b, rng)` → `Individual` . . . 17
- 8.5 `run(seed_prompts, problems, on_generation=None)` . . . 18

**9 `main.py` — Orchestration & Entry Point** **19**
- 9.1 The Three Functions . . . 19
  - 9.1.1 `save_checkpoint(run_dir, gen, population, stats)` . . . 19
  - 9.1.2 `evaluate_on_holdout(best_prompt, holdout)` → `float` . . . 19
  - 9.1.3 `main()` — The 7-Step Pipeline . . . 19
- 9.2 Output Files . . . 20

**10 Complete Worked Example** **21**
10.1 The Problem on Disk . . . . . 21
10.2 Fitness Evaluation: Step by Step . . . . . 21
10.3 After All 15 Seeds Are Evaluated . . . . . 22

**Glossary** **23**

# Chapter 1

# Introduction & Architecture

## 1.1 What the Project Does

A **Genetic Algorithm (GA)** automatically discovers the best **system prompt** — a paragraph of English instructions — for a Claude model that generates Python code. Each prompt is a *chromosome*; fitness is the average fraction of test cases the LLM-generated code passes on 100 APPS coding problems. Over 20 generations of selection, crossover, and mutation the GA evolves a prompt that solves more problems correctly.

**Key idea**

Manual prompt engineering is trial-and-error. This project automates it: the GA *searches* all possible English instruction paragraphs, using test-case pass rates as the fitness signal.

## 1.2 The Eight Files

| File | Lines | Role |
|---|---|---|
| `config.py` | 62 | Single source of truth for all hyperparameters and paths |
| `seeds.py` | 116 | 15 hand-crafted generation-0 chromosomes |
| `dataset.py` | 88 | Load APPS problems from disk; split into evolution / holdout |
| `evaluator.py` | 122 | Extract code from LLM text; run tests in isolated subprocess |
| `cost_tracker.py` | 178 | Log token counts; compute dollar cost; write JSON report |
| `llm.py` | 202 | All Claude API calls (code gen, crossover, mutation) |
| `ga.py` | 228 | The evolutionary loop: select, breed, evaluate, replace |
| `main.py` | 168 | Orchestrate the run; checkpoint; save results |

## 1.3 Module Dependency Graph

```
main.py ----imports----> config, dataset, evaluator, ga, llm
                         cost_tracker (singleton: tracker)
                         seeds        (SEED_PROMPTS list)

ga.py   ----imports----> config, evaluator, llm

llm.py  ----imports----> config
                         cost_tracker (singleton: tracker)
                         anthropic    (Anthropic API SDK)

evaluator.py   -------> stdlib only  (re, subprocess, sys)
dataset.py     -------> stdlib only  (json, random, sys, pathlib)
cost_tracker.py ------> stdlib only  (json, time, dataclasses, datetime, pathlib)
```

```
seeds.py        -------> no imports
```

**Network boundary**

`llm.py` is the **only** file that touches the network. All other files are pure computation or local disk I/O, so they can be tested without an API key.

## 1.4 Two-Model Architecture

| Role | Model | Temp. | Purpose |
|---|---|---|---|
| Target (subject) | `claude-haiku-4-5-20251001` | 0.0 | Generate code; fitness eval |
| Optimizer (tool) | `claude-sonnet-4-6` | 0.7 | Crossover and mutation |

Temperature 0.0 on the target model ensures the same prompt always produces the same code on the same problem, making fitness scores comparable across generations. Temperature 0.7 on the optimizer introduces creativity for blending and mutating prompts.

## 1.5 Master Call-Flow (one generation)

```
main.main()
  |-- dataset.load_problems(APPS_TRAIN)       -> list[dict]  (~2346 problems)
  |-- dataset.split(problems, 100, 200, 42)   -> (evolution[100], holdout[200],
      rest)
  |
  +-- ga.run(SEED_PROMPTS, evolution_pool, on_generation=save_checkpoint)
        |
        | [GEN 0] for each of 15 seeds:
        |   compute_fitness(ind, problems)          [100 API calls per seed]
        |     -> llm.generate_code(prompt, q, fn)   -> raw LLM text
        |       -> llm._call("fitness", Haiku, T=0) -> resp.content[0].text
        |           -> tracker.record(tokens)
        |     -> evaluator.extract_code(response)   -> code string
        |     -> evaluator.run_tests(code, ...)      -> float [0,1]
        |
        | [GENS 1..20] for each of 13 new offspring:
        |   tournament_select(population)  x2        -> parent_a, parent_b
        |   breed(parent_a, parent_b)
        |     -> llm.crossover(a, b)    [80%]         -> merged prompt
        |     -> llm.mutate_inject(p)   [25%]         -> prompt + new strategy
        |     -> llm.mutate_delete(p)   [15%]         -> prompt - weak sentence
        |     -> llm.mutate_rephrase(p)[10%]          -> prompt (rephrased)
        |     -> Individual(prompt, uid, parents, ops)  [fitness = -1]
        |   compute_fitness(child, problems)          [100 API calls per child]
        |   save_checkpoint(run_dir, gen, population) -> generation_NN.json
        |
        +-- returns (best_Individual, history[21 dicts])
  |
  |-- evaluate_on_holdout(best.prompt, holdout[200])  [200 API calls]
  +-- tracker.flush(extra)   -> Code/Costs/cost_TIMESTAMP.json
```

**API call scale**

One full run makes approximately:
$15 \times 100 = 1{,}500$ fitness calls (gen 0) + $20 \times 13 \times 100 = 26{,}000$ (gens 1–20) + 200 (holdout) = **27,700 Haiku calls** + 800 Sonnet calls for operators.

# Chapter 2

# config.py — Hyperparameters

## 2.1 Purpose & Design

Every number and path lives here. No other file contains hard-coded values. All constants are read at import time; changing `config.py` affects the whole system.

## 2.2 Key Constants Explained

Listing 2.1: config.py (abridged)

```python
from pathlib import Path
from dotenv import load_dotenv
load_dotenv()  # reads .env for ANTHROPIC_API_KEY

ROOT          = Path(__file__).parent.parent      # Project/
APPS_TRAIN    = ROOT / "Dataset Examples/APPS/train"
RESULTS_DIR   = Path(__file__).parent / "results"    # Code/results/

EVOLUTION_SIZE = 100    # problems used as fitness benchmark
HOLDOUT_SIZE   = 200    # problems reserved for final test
RANDOM_SEED    = 42

POPULATION_SIZE    = 15
GENERATIONS        = 20
ELITISM_COUNT      = 2    # top-2 survive unchanged every generation
TOURNAMENT_SIZE    = 3    # pick best of 3 random draws for each parent

CROSSOVER_RATE     = 0.8     # probability crossover fires per offspring
MUT_INJECT_PROB    = 0.25    # probability inject mutation fires (independent)
MUT_DELETE_PROB    = 0.15
MUT_REPHRASE_PROB = 0.10

TARGET_MODEL        = "claude-haiku-4-5-20251001"
TARGET_TEMPERATURE = 0.0       # deterministic; repeatable fitness
OPTIMIZER_MODEL        = "claude-sonnet-4-6"
OPTIMIZER_TEMPERATURE = 0.7    # creative; varied crossover/mutation
MAX_CODE_TOKENS      = 1024
MAX_OPERATOR_TOKENS = 600
EXEC_TIMEOUT = 5    # seconds before subprocess is killed
```

**Elitism prevents regression**

`ELITISM_COUNT=2` guarantees the top 2 prompts always pass unchanged into the next generation. Without elitism, an excellent prompt found in generation 5 could be lost if all its offspring happen to score worse.

**Mutations are independent**

Each of the three mutation probabilities is checked with a separate `rng.random()` call, so all three can fire on the same offspring (probability $0.25 \times 0.15 \times 0.10 = 0.00375$), or none can fire ($0.75 \times 0.85 \times 0.90 = 0.574$). This gives $2^3 = 8$ possible mutation combinations per offspring.

# Chapter 3

# `seeds.py` — Initial Population

`seeds.py` defines `SEED_PROMPTS`: a Python `list[str]` of 15 diverse instruction paragraphs. Each is 2–4 sentences emphasising a different coding strategy:

| # | Strategy | Core emphasis |
|---|---|---|
| 1 | Structured analysis | Identify types and edge cases before coding |
| 2 | Competitive programmer | Find the algorithm pattern first |
| 3 | Return-type focus | Match exact return type and format |
| 4 | Step-by-step reasoning | Four-step process: understand, identify, handle, code |
| 5 | Example-driven | Trace examples manually before implementing |
| 6 | Minimal / precise | Simplest correct solution; no extra code |
| 7 | Edge-case defender | Enumerate: empty, single, zero, negative, max |
| 8 | Algorithm-first | Choose data structure before writing a line |
| 9 | Output-format guard | Verify return type (int/list/str/bool/tuple) |
| 10 | Pseudocode-then-code | English sketch first, then translate |
| 11 | Constraint-aware | Match time complexity to stated constraints |
| 12 | Defensive / robust | Assume worst-case valid input |
| 13 | Readability-first | Meaningful names; correct beats clever |
| 14 | Verify-against-examples | Mental verification before finalising |
| 15 | Direct and focused | Transformation-first; no extra output |

The file ends with `assert len(SEED_PROMPTS) == 15` — if a seed is accidentally added or removed the program crashes immediately on import, before any API calls are made.

In `ga.run()`, these 15 strings become `Individual` objects:

```
population = [Individual(prompt=p) for p in seed_prompts]
# Each Individual: prompt=<seed>, fitness=-1.0, uid=<random 8-char hex>,
#                  parent_a="", parent_b="", operators=""
```

# Chapter 4

# dataset.py — Loading & Splitting Problems

## 4.1 load_problems(apps_train_dir) → list[dict]

**load_problems(apps_train_dir) -> list[dict]**

Called by: main.main() [main.py:75]
Calls: Path.iterdir(), json.loads(), Path.read_text()

Iterates every subdirectory of `APPS/train/`. Each subdirectory is one problem. Three files are required: `metadata.json`, `input_output.json`, `question.txt`. A five-condition filter keeps only problems that are: (1) `difficulty == "introductory"`, (2) have a `fn_name` field, (3) sourced from `codewars.com`, (4) have non-empty `inputs`, and (5) have non-empty `outputs`.

**Why Codewars only?** Codewars problems use standalone functions (`def grader(score): return ...`), giving a consistent I/O format. LeetCode uses `class Solution` methods which complicates code extraction. Inputs are parsed Python argument lists; outputs are `[return_value]` (single-element list).

Each passing problem produces:

```
{
    "id":       "3715",
    "question": "Complete the function that takes an array of words...",
    "fn_name":  "nth_char",
    "inputs":   [ [["yoda","best","has"]], [["a","bc","def"]] ],
    "outputs":  [ ["oes"], ["abd"] ]
    # inputs[i] = list of args  => fn_name(*inputs[i])
    # outputs[i] = [return_value]  => [result] == outputs[i]
}
```

## 4.2 split(problems, evolution_size, holdout_size, seed=42) → (list, list, list)

**split(...) -> (evolution[100], holdout[200], rest)**

Called by: main.main() [main.py:82]
Calls: random.Random(seed), list.copy(), rng.shuffle()

```
rng = random.Random(seed)    # isolated RNG, does not affect global random state
shuffled = problems.copy()   # never modify the caller's list
rng.shuffle(shuffled)        # deterministic: seed 42 always gives the same order

evolution = shuffled[:100]          # the GA's training set
holdout   = shuffled[100:300]       # reserved for final generalisation test
rest      = shuffled[300:]          # unused buffer
```

The fixed seed is critical: if the 100 evolution problems changed between generations, fitness scores from generation 3 and generation 7 would not be comparable.

**Train / test split**

The GA "trains" on 100 problems (evolution pool). The best evolved prompt is then evaluated on 200 never-seen holdout problems. If holdout fitness is much lower than evolution fitness, the prompt has overfitted to the 100 training problems.

# Chapter 5

# evaluator.py — Code Extraction & Sandboxed Testing

## 5.1 Overview

Two responsibilities: (1) strip Python code out of an LLM's raw text response, and (2) run that code in an isolated subprocess and return a partial-credit score.

## 5.2 extract_code(response: str) → str

**extract_code(response) -> str**

Called by: ga.compute_fitness() [ga.py:68]; main.evaluate_on_holdout() [main.py:54]
Calls: re.search() (at most twice)

Tries three strategies in order, returning on first success:

```
# 1. Tagged python fence
m = re.search(r"```python\s*\n([\s\S]*?)```", response)
if m: return m.group(1).strip()

# 2. Any fence
m = re.search(r"```\s*\n?([\s\S]*?)```", response)
if m: return m.group(1).strip()

# 3. Assume raw code
return response.strip()
```

Example: input `"Here's my answer:\n"`python\ndef nth_char(words):\n ...\n"`"` → output `"def nth_char(words):\n ..."`.

## 5.3 has_function(code: str, fn_name: str) → bool

**has_function(code, fn_name) -> bool**

Called by: run_tests() [evaluator.py:97]
Calls: re.search(), re.escape()

```
return bool(re.search(rf"\bdef\s+{re.escape(fn_name)}\s*\(", code))
# \b = word boundary, \s+ = spaces, \s* = optional space before (
# Guards against wrong function name (NameError), wrong class (method).
# If False, run_tests() returns 0.0 immediately without launching subprocess.
```

## 5.4 run_tests(code, fn_name, inputs, outputs, timeout=5) → float

**run_tests(...) -> float in [0.0, 1.0]**

Called by: ga.compute_fitness() [ga.py:69]; main.evaluate_on_holdout() [main.py:56]
Calls: has_function(), _build_script(), subprocess.run()

**How it works end-to-end:**

```
run_tests(code, fn_name, inputs, outputs, timeout=5)
  |
  +-- if not code or not has_function(code, fn_name):
  |       return 0.0    # early-out: no code or wrong function name
  |
  +-- script = _build_script(code, fn_name, inputs, outputs)
  |       Produces a complete Python script (see below)
  |
  +-- proc = subprocess.run(
  |       [sys.executable, "-c", script],
  |       capture_output=True, text=True, timeout=timeout)
  |       # Fully isolated process; if it crashes, GA is unaffected
  |
  +-- results = [int(line[len("__R__:"):])
  |               for line in proc.stdout.splitlines()
  |               if line.startswith("__R__:")]
  |       # Only sentinel lines are parsed; stray print() calls are ignored
  |
  +-- if not results: return 0.0   # script crashed before any test ran
  +-- return sum(results) / len(inputs)   # partial credit
        # denominator = TOTAL tests, not just those that produced output
        # e.g. 3 pass out of 4 = 0.75 even if script crashed mid-way
```

**The test harness script** assembled by `_build_script()` looks like this for an `nth_char` problem:

Listing 5.1: Script produced by _build_script() – executed in subprocess

```python
import sys
sys.set_int_max_str_digits(1_000_000)

def nth_char(words):                # <-- LLM-generated code pasted verbatim
    return ''.join(word[i] if i < len(word) else '#'
                   for i, word in enumerate(words, 1))

def __run__(inp, expected):
    try:
        result = nth_char(*inp)     # call the generated function
        # outputs are stored as [return_value], so wrap result in list:
        sys.stdout.write("__R__:" + ("1" if [result] == expected else "0") + "\n"
            )
        sys.stdout.flush()
    except Exception:               # catches NameError, TypeError, etc.
        sys.stdout.write("__R__:0\n")
        sys.stdout.flush()

__run__([["yoda","best","has"]], ["oes"])   # one line per test case
__run__([["a","bc","def"]], ["abd"])
```

**Why [result] == expected?**

APPS Codewars outputs are stored as `[return_value]`. Wrapping the actual result in `[result]` makes the comparison format match: `["oes"] == ["oes"]` is `True`; `"oes" == ["oes"]` would

always be `False`.

**Why a subprocess?**

Generated code is untrusted. Direct import could crash the GA process (via `sys.exit()` or infinite loop with no timeout). The subprocess is killed after 5 seconds; the GA continues regardless.

**Division by `len(inputs)`, not `len(results)`**

If generated code crashes after 2 of 4 tests, only 2 `__R__:` lines appear. But the denominator is still 4 (total tests), not 2 (completed tests). This fairly penalises code that fails partway through.

# Chapter 6

# cost_tracker.py — API Cost Logging

## 6.1 Purpose & Location

Every Claude API call has a dollar cost. This module logs token counts, computes costs from official Anthropic pricing, and writes a JSON breakdown to `Code/Costs/cost_TIMESTAMP.json` after each run.

```
COSTS_DIR = Path(__file__).parent / "Costs"    # Code/Costs/
```

## 6.2 CallRecord — One API Call's Data

**@dataclass CallRecord**

Created by: CostTracker.record() [cost_tracker.py:73]
Stores: phase (str), model (str), input_tokens (int), output_tokens (int)
Computes on-the-fly: input_cost, output_cost, total_cost (via PRICING dict)

```
PRICING = {    # (input_$/MTok, output_$/MTok)
    "claude-haiku-4-5-20251001": (1.00,  5.00),
    "claude-sonnet-4-6":         (3.00, 15.00),
    ...
}

@property
def total_cost(self) -> float:
    rate_in, rate_out = PRICING.get(self.model, (0, 0))
    return (self.input_tokens * rate_in
            + self.output_tokens * rate_out) / 1_000_000
# Example: 287 input tokens + 52 output tokens on Haiku 4.5:
#    = (287*1.00 + 52*5.00) / 1e6 = $0.000547 per call
```

## 6.3 CostTracker — The Singleton Accumulator

**class CostTracker**

Module-level singleton: tracker = CostTracker() [line 197]
Imported by llm.py as: from cost_tracker import tracker

```
CostTracker.record(phase, model, input_tokens, output_tokens)
  Called by: llm._call() after every successful API response
  Action: appends CallRecord to self.calls (in-memory log)

CostTracker._by_phase() -> dict
  Groups self.calls by phase ("fitness", "crossover", "mutate_inject", ...)
  Returns totals per phase: calls, tokens, cost

CostTracker._by_model() -> dict
  Groups self.calls by model ID
  Returns totals per model
```

```
CostTracker.flush(extra=None) -> Path
  Called by: main.main() [main.py:154]  -- once per run, at the very end
  Action:
    - COSTS_DIR.mkdir(parents=True, exist_ok=True)   # create if needed
    - assembles JSON report: totals + _by_phase() + _by_model() + pricing_used
    - writes Code/Costs/cost_TIMESTAMP.json
    - returns Path of the written file
```

**Singleton pattern**

Python caches imported modules. `from cost_tracker import tracker` in both `llm.py` and `main.py` gets the *exact same* `CostTracker` instance. So every `_call()` adds to the same list, and `flush()` sees the complete history for the whole run.

# Chapter 7

# llm.py — All Claude API Calls

## 7.1 Overview

The sole interface to the Anthropic API. All five public functions route through the private `_call()` wrapper, which handles authentication, retries, and cost logging.

```
Public API:
  generate_code(system_prompt, question, fn_name)  -> str  [raw LLM text]
  crossover(prompt_a, prompt_b)                    -> str  [merged prompt]
  mutate_inject(prompt)                            -> str  [prompt + strategy]
  mutate_delete(prompt)                            -> str  [prompt - sentence]
  mutate_rephrase(prompt)                          -> str  [prompt rephrased]

All call through:
  _call(phase, model, temperature, max_tokens, system, user) -> str
    which calls _get_client() once (lazy init) then client.messages.create(...)
```

## 7.2 `_call()` — The Core Wrapper

**`_call(phase, model, temperature, max_tokens, system, user, retries=3) -> str`**

Called by: all five public functions
Calls: _get_client(), client.messages.create(), tracker.record(), time.sleep()

Listing 7.1: _call() – retry and logging logic

```python
def _call(phase, model, temperature, max_tokens, system, user, retries=3):
    client = _get_client()    # returns cached anthropic.Anthropic(api_key=...)
    for attempt in range(retries):
        try:
            resp = client.messages.create(
                model=model, max_tokens=max_tokens,
                temperature=temperature, system=system,
                messages=[{"role": "user", "content": user}],
            )
            tracker.record(phase=phase, model=model,  # log tokens + cost
                input_tokens=resp.usage.input_tokens,
                output_tokens=resp.usage.output_tokens)
            return resp.content[0].text                # the assistant's response

        except anthropic.RateLimitError:
            wait = 2 ** attempt * 5    # 5s, 10s, 20s (exponential backoff)
            time.sleep(wait)
        except anthropic.APIError as e:
            if attempt == retries - 1: return ""       # give up; return empty
            time.sleep(2)
    return ""    # all retries exhausted
```

**What happens when `_call` returns `""`**

An empty response propagates as: `extract_code("")` → `""`, then `run_tests("", ...)` → 0.0 immediately (empty-code guard). The problem scores 0; the run continues. One failed API call

does not crash the GA.

## 7.3 generate_code() — Fitness Evaluation

**generate_code(system_prompt, question, fn_name) -> str**

Called by: ga.compute_fitness() [ga.py:63]; main.evaluate_on_holdout() [main.py:52]
Calls: _call(phase="fitness", model=TARGET_MODEL, temperature=0.0, max_tokens=1024)

```
def generate_code(system_prompt, question, fn_name):
    user_msg = f"{question}\n\nWrite a Python function named `{fn_name}`."
    return _call(
        phase="fitness",
        model=config.TARGET_MODEL,       # Haiku 4.5
        temperature=config.TARGET_TEMPERATURE,  # 0.0 = deterministic
        max_tokens=config.MAX_CODE_TOKENS,      # 1024
        system=system_prompt,    # THE CHROMOSOME -- the prompt being evolved
        user=user_msg,           # problem-specific: changes per problem
    )
# Returns raw LLM text. Code extraction happens in evaluator.extract_code().
# The separation is intentional: system = general strategy; user = specific
    problem.
```

## 7.4 crossover() — LLM-Mediated Merge

**crossover(prompt_a, prompt_b) -> str**

Called by: ga.breed() [ga.py:117] when rng.random() < 0.8
Calls: _call(phase="crossover", model=OPTIMIZER_MODEL, temperature=0.7, max_tokens=600)

The Optimizer (Sonnet 4.6) receives a meta-prompt asking it to *blend* the two parents into one coherent paragraph — explicitly forbidden from simple concatenation.

```
_CROSSOVER_USER = """
Combine these two coding-instruction prompts into one coherent prompt.
Preserve the strongest strategies from each parent.
Do not simply concatenate -- produce a unified instruction paragraph.

--- Parent A ---
{parent_a}

--- Parent B ---
{parent_b}
"""
# Example offspring:
# "Approach the problem like a competitive programmer: identify the algorithm
#  pattern. Before coding, explicitly consider boundary conditions: empty
#  input, zero, negative values. Write the most direct, correct solution."
```

## 7.5 mutate_inject(), mutate_delete(), mutate_rephrase()

All three call `_call()` with `OPTIMIZER_MODEL` at temperature 0.7 and `MAX_OPERATOR_TOKENS=600`. The phase label differs for cost tracking.

```
mutate_inject(prompt) -> str             [phase="mutate_inject"]
  Meta-prompt: "Add one new, specific coding strategy."
  Examples: 'check for off-by-one errors', 'consider time complexity'
  Returns: prompt with one new sentence added.

mutate_delete(prompt) -> str             [phase="mutate_delete"]
  Edge case FIRST: if len(re.split(r"(?<=[.!?])\s+", prompt)) <= 1:
      return prompt   # only one sentence; skip API call, return unchanged
  Meta-prompt: "Remove the sentence that contributes the least."
  Returns: leaner prompt with weakest sentence removed.

mutate_rephrase(prompt) -> str           [phase="mutate_rephrase"]
  Meta-prompt: "Rephrase using different words, keep same meaning."
  Returns: same strategies, different wording.
```

# Chapter 8

# ga.py — The Genetic Algorithm

## 8.1 Individual — The Chromosome Dataclass

```python
@dataclass
class Individual:
    prompt:    str              # the system prompt (the chromosome)
    fitness:   float = -1.0    # -1.0 = sentinel: not yet evaluated
    uid:       str = field(default_factory=lambda: uuid.uuid4().hex[:8])
    parent_a:  str = ""         # uid of first parent  (empty for seeds)
    parent_b:  str = ""         # uid of second parent (empty for seeds)
    operators: str = ""         # e.g. "crossover+inject" or "copy+rephrase"
```

## 8.2 compute_fitness(individual, problems) → float

**compute_fitness(individual, problems) -> float in [0.0, 1.0]**

Called by: run() – 15 times at gen 0; 13 times per generation (gens 1–20)
Calls: llm.generate_code(), evaluator.extract_code(), evaluator.run_tests()

```
compute_fitness(individual, problems):
  total = 0.0
  for prob in problems:       # 100 iterations

    response = llm.generate_code(         [llm.py:77]
                individual.prompt,
                prob["question"],
                prob["fn_name"])
              -> _call("fitness", Haiku, T=0.0, max=1024, ...)
              -> tracker.record(tokens)
              -> returns raw LLM text

    code = evaluator.extract_code(response)   [evaluator.py:24]
              -> returns code string

    score = evaluator.run_tests(              [evaluator.py:84]
               code, prob["fn_name"],
               prob["inputs"], prob["outputs"], timeout=5)
              -> has_function() check
              -> _build_script() assembles harness
              -> subprocess.run() executes
              -> parse __R__: lines
              -> returns float [0.0, 1.0]

    total += score

  return total / len(problems)   # mean partial credit across 100 problems
```

**Expensive operation**

One call to `compute_fitness()` makes **100 Haiku API calls**. At ~$0.0005 per call, evaluating one individual costs ~$0.05. Evaluating all 275 individuals in a full run costs ~$13–25.

## 8.3 tournament_select(population, rng) → Individual

**tournament_select(population, rng) -> Individual**

Called by: run() – twice per offspring [ga.py:199–200]
Calls: rng.sample(population, 3), max(..., key=fitness)

```
contestants = rng.sample(population, config.TOURNAMENT_SIZE)  # 3 draws, no
    replace
return max(contestants, key=lambda ind: ind.fitness)
# The fittest of 3 random draws becomes a parent.
# Weaker individuals can win if stronger ones are not drawn.
# This balances exploitation (best usually wins) with exploration.
```

## 8.4 breed(parent_a, parent_b, rng) → Individual

**breed(parent_a, parent_b, rng) -> Individual (fitness=-1.0)**

Called by: run() [ga.py:201] – 13 times per generation
Calls: llm.crossover() OR rng.choice(), then optionally llm.mutate_inject/delete/rephrase()

```
breed(parent_a, parent_b, rng):

  [STEP 1 -- Crossover or copy]
  if rng.random() < 0.80:
      prompt = llm.crossover(parent_a.prompt, parent_b.prompt)
               -> _call("crossover", Sonnet, T=0.7, max=600, ...)
               -> returns merged prompt string
      ops = ["crossover"]
  else:
      prompt = rng.choice([parent_a.prompt, parent_b.prompt])
      ops = ["copy"]

  [STEP 2 -- Inject? (independent roll)]
  if rng.random() < 0.25:
      prompt = llm.mutate_inject(prompt)
               -> _call("mutate_inject", Sonnet, T=0.7, max=600, ...)
      ops.append("inject")

  [STEP 3 -- Delete? (independent roll)]
  if rng.random() < 0.15:
      prompt = llm.mutate_delete(prompt)   # may skip _call if 1 sentence
      ops.append("delete")

  [STEP 4 -- Rephrase? (independent roll)]
  if rng.random() < 0.10:
      prompt = llm.mutate_rephrase(prompt)
               -> _call("mutate_rephrase", Sonnet, T=0.7, max=600, ...)
      ops.append("rephrase")

  return Individual(
      prompt=prompt,
      parent_a=parent_a.uid,          # genealogy tracking
      parent_b=parent_b.uid,
      operators="+".join(ops),        # e.g. "crossover+inject" or "copy"
  )
  # fitness = -1.0 (sentinel; run() calls compute_fitness() separately)
```

## 8.5 run(seed_prompts, problems, on_generation=None)

**run(...) -> (best_Individual, history: list[dict])**

Called by: main.main() [main.py:120]
Calls: compute_fitness(), tournament_select(), breed(), on_generation callback

```
run(seed_prompts, problems, on_generation):

  rng = random.Random(42)    # seeded; reproducible across runs
  population = [Individual(prompt=p) for p in seed_prompts]  # 15 individuals

  [GEN 0] for each ind in population:
      ind.fitness = compute_fitness(ind, problems)    # 15 x 100 = 1500 API calls

  [GENS 1..20] for gen in range(1, 21):

      population.sort(by fitness, descending)

      stats = {generation: gen-1, best, mean, worst, best_uid, best_prompt}
      history.append(stats)       # records state of PREVIOUS generation

      if on_generation:
          on_generation(gen-1, population, stats)
          # = main.save_checkpoint(run_dir, gen-1, pop, stats)
          # Writes: results/run_*/generation_NN.json

      new_pop = population[:2]    # ELITISM: top 2 carried unchanged

      while len(new_pop) < 15:    # breed 13 new offspring
          p1 = tournament_select(population, rng)
          p2 = tournament_select(population, rng)
          child = breed(p1, p2, rng)               # 0-4 Sonnet calls
          new_pop.append(child)

      for ind in new_pop[2:]:     # evaluate only the 13 NEW offspring
          ind.fitness = compute_fitness(ind, problems)   # 13 x 100 = 1300 calls

      population = new_pop

  [FINAL]
  population.sort(by fitness, descending)
  history.append({generation: 20, ...})   # 21st record

  return (population[0], history)         # best prompt + 21-entry history
```

**Generational replacement**

The entire population is replaced each generation (except the 2 elites). This is **full generational replacement**: only the top 2 are guaranteed to survive. It creates strong selection pressure and fast convergence at the cost of some diversity.

**History records**

`history` has 21 entries (generations 0–20). Each entry has: `generation`, `best`, `mean`, `worst`, `best_uid`, `best_prompt`. Generations 0–19 are recorded at the *start* of the next breeding step; generation 20 is recorded after the loop ends.

# Chapter 9

# `main.py` — Orchestration & Entry Point

## 9.1 The Three Functions

### 9.1.1 `save_checkpoint(run_dir, gen, population, stats)`

**`save_checkpoint(...)`**

Called by: the on_generation lambda in main() [main.py:123]
Calls: json.dumps(), Path.write_text()
Output: results/run_*/generation_NN.json

Saves the full population state (all 15 prompts, fitnesses, parent UIDs, operators) after each generation. If the run crashes at generation 15, at most one generation of API spending is lost.

### 9.1.2 `evaluate_on_holdout(best_prompt, holdout)` → `float`

**`evaluate_on_holdout(best_prompt, holdout) -> float`**

Called by: main() [main.py:139]
Calls: llm.generate_code(), evaluator.extract_code(), evaluator.run_tests()
Same pipeline as compute_fitness() but over 200 holdout problems.

Structurally identical to `compute_fitness()` but runs on the 200 held-out problems and prints progress every 50. Returns the mean partial-credit score.

### 9.1.3 `main()` — The 7-Step Pipeline

```
main():

  [1] Pre-flight checks
      if not config.ANTHROPIC_API_KEY: sys.exit(1)
      if not config.APPS_TRAIN.exists(): sys.exit(1)
      # Fails fast BEFORE any expensive operations

  [2] Load and split dataset
      all_problems = dataset.load_problems(config.APPS_TRAIN)    # ~2346 dicts
      evolution_pool, holdout, rest = dataset.split(
          all_problems, 100, 200, seed=42)

  [3] Create timestamped run directory
      run_dir = config.RESULTS_DIR / "run_YYYYMMDD_HHMMSS"
      run_dir.mkdir(parents=True, exist_ok=True)
      Writes: config.json           (hyperparameter snapshot)
              evolution_ids.json    (which 100 problem IDs)

  [4] Run GA
      best, history = ga.run(
          seed_prompts=SEED_PROMPTS,
          problems=evolution_pool,
          on_generation=lambda gen, pop, stats:
              save_checkpoint(run_dir, gen, pop, stats))
      # Takes hours; checkpoints every generation
```

```
[5] Save results
    (run_dir / "history.json").write_text(json.dumps(history))
    (run_dir / "best_prompt.txt").write_text(best.prompt)

[6] Holdout evaluation
    holdout_fitness = evaluate_on_holdout(best.prompt, holdout)
    # 200 more Haiku calls; measures generalisation to unseen problems

[7] Save cost report
    tracker.flush(extra={fitnesses, run_dir, ...})
    -> Writes: Code/Costs/cost_TIMESTAMP.json
```

## 9.2 Output Files

| File | Contents |
|---|---|
| `run_*/config.json` | Exact hyperparameters (reproducibility) |
| `run_*/evolution_ids.json` | The 100 problem IDs used as the fitness pool |
| `run_*/generation_NN.json` | Full population per generation (genealogy) |
| `run_*/history.json` | Best / mean / worst fitness, 21 records |
| `run_*/best_prompt.txt` | The evolved prompt (main artefact) |
| `run_*/summary.json` | Final fitnesses (evolution + holdout), runtime |
| `Costs/cost_*.json` | Token counts and dollar cost breakdown |

# Chapter 10

# Complete Worked Example

This chapter traces one problem from disk to a fitness score using realistic data.

## 10.1 The Problem on Disk

```
# metadata.json:  {"difficulty": "introductory", "url": "https://codewars.com/..."}
# question.txt:   "Complete the function that takes an array of words.
#                  Concatenate the nth letter from each word using the
#                  enumerating order (1-indexed). Use '#' if too short.
#                  nth_char(['yoda','best','has']) == 'oes'"
# input_output.json:
{
    "fn_name": "nth_char",
    "inputs":  [ [["yoda","best","has"]], [["a","bc","def"]] ],
    "outputs": [ ["oes"], ["abd"] ]
}
```

After `load_problems()` the dict is: `{"id":"3715","question":"...","fn_name":"nth_char","inputs":[`

## 10.2 Fitness Evaluation: Step by Step

**Step 1 — `llm.generate_code()` is called:**

```
# system = "Think like a competitive programmer..."  (the chromosome)
# user   = "Complete the function that takes an array of words...\n\n"
#          "Write a Python function named `nth_char`."
# -> _call(phase="fitness", model="claude-haiku-4-5-20251001", temperature=0.0, ...)
# -> tracker.record("fitness", "claude-haiku-4-5-20251001", 287, 52)
#    input_cost = 287*1.00/1e6 = $0.000287; output_cost = 52*5.00/1e6 = $0.000260
# Returns raw text:
raw = "```python\ndef nth_char(words):\n    return ''.join(\n        word[i] if i < len(word) else '#'\n        for i, word in enumerate(words, 1))\n```"
```

**Step 2 — `evaluator.extract_code()` strips the markdown:**

```
code = "def nth_char(words):\n    return ''.join(\n        word[i] if i < len(word) else '#'\n        for i, word in enumerate(words, 1))"
```

**Step 3 — `evaluator.run_tests()` executes the harness:**

```
# has_function check: re.search(r"\bdef\s+nth_char\s*\(", code) -> True

# _build_script() produces (simplified):
# "...def nth_char(words): ...\n"
# "__run__([['yoda','best','has']], ['oes'])\n"
# "__run__([['a','bc','def']], ['abd'])\n"

# subprocess.run([python3, "-c", script], timeout=5)
#   Test 1: nth_char(['yoda','best','has'])
#     enumerate(words, 1): (1,"yoda"),(2,"best"),(3,"has")
#     word[1]='o', word[2]='e', word[3]='s'  => "oes"
#     [result]==["oes"] -> True  -> stdout: "__R__:1"
#   Test 2: nth_char(['a','bc','def'])
```

```
#      word[1]='a', word[2]='b', word[3]='d'  => "abd"
#      ["abd"]==["abd"] -> True  -> stdout: "__R__:1"

# proc.stdout = "__R__:1\n__R__:1\n"
# results = [1, 1]
# score = sum([1,1]) / len(inputs) = 2/2 = 1.0
```

**Step 4 — Score accumulates in `compute_fitness()`:**

This problem contributes 1.0 to `total`. After all 100 problems, `fitness = total / 100`. If the competitive-programmer prompt scores 1.0 on 73 problems and partial credit on others, the overall fitness might be 0.7312. This is stored as `individual.fitness = 0.7312`.

## 10.3 After All 15 Seeds Are Evaluated

```
Gen 0 population (sorted by fitness, example):
  uid=a3f7c2b1  fitness=0.7312  prompt="Think like competitive programmer..."
  uid=c9b2d3f0  fitness=0.7108  prompt="You are an expert Python programmer..."
  uid=e1f4a2b8  fitness=0.6893  prompt="Approach step by step..."
  ...
  uid=d7e1f3c2  fitness=0.5920  prompt="Write clean, readable Python code..."

Stats for gen 0: best=0.7312, mean=0.6841, worst=0.5920
save_checkpoint() writes: results/run_*/generation_00.json

Elitism: new_pop = [a3f7c2b1, c9b2d3f0]   (top 2 kept unchanged)

Breed 13 offspring:
  tournament draws e.g. p1=a3f7c2b1, p2=c9b2d3f0
  rng.random()=0.61 < 0.80 => crossover
      llm.crossover(prompt_a, prompt_b) -> Sonnet merges them
  rng.random()=0.18 < 0.25 => inject
      llm.mutate_inject(merged) -> adds "check time complexity"
  rng.random()=0.73 >= 0.15 => no delete
  rng.random()=0.84 >= 0.10 => no rephrase
  child: operators="crossover+inject", fitness=-1.0

  compute_fitness(child, problems)  -> 100 Haiku calls -> e.g. fitness=0.7589

... repeat for all 13 offspring ...
After evaluation: population = [2 elites] + [13 evaluated offspring]
Gen 1 complete.  20 generations in total.
```

# Glossary

**Chromosome** A candidate solution in a GA. Here: a system prompt string, stored as an `Individual` object.

**Crossover** Genetic operator combining two parents into one offspring. Here: an LLM semantically blends two parent prompts. Fires with probability 0.8.

**Elitism** Copying the top $k$ individuals unchanged into the next generation (`ELITISM_COUNT=2`). Prevents the best solution from ever being lost.

**Evolution pool** The fixed 100 APPS problems used as the fitness benchmark. Chosen by deterministic shuffle with seed 42.

**Exponential backoff** Waiting 5s, 10s, 20s between retries on rate-limit errors (`wait = 2**attempt * 5` in `_call()`).

**Fitness** Mean partial-credit score across 100 problems. Range: [0.0, 1.0]. Computed by `ga.compute_fitness()`.

**`fn_name`** The Python function name the LLM must write, from each problem's `input_output.json`.

**Generational replacement** The entire population is replaced each generation (except elites), creating strong selection pressure.

**Held-out set** 200 problems never seen during evolution. Used to test generalisation of the final evolved prompt.

**Individual** One GA population member: a prompt with fitness, uid, parent uids, and operator history.

**Lazy init** `_get_client()` only creates the `anthropic.Anthropic` object on first use, not at module import time.

**Meta-prompt** A prompt sent to the Optimizer LLM instructing it to modify another prompt (used in crossover and all three mutation operators).

**Mutation** Modifying a single chromosome. Three independent types: inject (25%), delete (15%), rephrase (10%).

**Partial credit** Score = (tests passed) / (total tests). Provides a smooth gradient vs. binary pass/fail.

**Sentinel** The `__R__:` prefix tags test results in subprocess stdout, separating them from any stray `print()` calls in generated code.

**Singleton** The `tracker = CostTracker()` instance at module level is shared by all importers (`llm.py` and `main.py`).

**System prompt** English instructions sent before the user message, shaping LLM behaviour. The GA chromosome.

**Temperature** LLM randomness parameter. 0.0 = deterministic (target model); 0.7 = creative (optimizer model).

**Tournament selection**
: Sample $k = 3$ individuals at random; return the fittest. Balances exploitation with exploration.

**Two-model architecture**
: Haiku 4.5 (cheap, deterministic) is optimised; Sonnet 4.6 (capable, creative) performs the genetic operators.